



Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat

Department of Artificial Intelligence

Intelligent Multi-Agent and Expert Systems (AI401), Semester VII

U23AI052 Lab 6 Submission

Experiment 6

Coordination Mechanisms — Centralized and Decentralized Task Allocation

Aim

To allocate a set of tasks among a set of agents under two coordination mechanisms — a central coordinator holding global state, and agents negotiating locally by announcing intentions — and to compare the two by solution quality, number of messages exchanged and behaviour when an agent fails.

Theory

When several agents share a set of tasks, someone must decide which agent does what. Under centralized coordination every agent reports its state to one coordinator, which holds the complete cost matrix, searches the space of assignments and returns a decision. The result is globally optimal and the message count is linear in the number of agents, but the search space grows as the factorial of that number, and the coordinator is a single point of failure: if it stops, nothing is allocated at all.

Under decentralized coordination there is no such authority. Each unassigned agent evaluates the tasks it can see, announces the one it intends to take, and a fixed conflict rule — lowest cost, ties broken by identifier — decides who keeps it; the agents that lost re-evaluate what remains and the round repeats. Every agent uses only local information, so the outcome is a greedy matching that can be far from optimal: an agent that claims a task early may leave the only remaining task for a distant neighbour. In exchange, no agent is indispensable, and the price is paid in messages, since each announcement is broadcast to everyone. Measuring that trade-off — optimality against messages and robustness — is the purpose of this experiment; the negotiation protocols of Experiment 7 refine the same bidding step.

Algorithm

Algorithm 1 — Centralized allocation

1. Place the agents and the tasks at fixed positions on a grid and build the cost matrix, taking the cost of an agent performing a task to be the Manhattan distance between them; this matrix is what every agent would send to the coordinator.
2. Enumerate the assignments of distinct tasks to agents, evaluate the total cost of each, and keep the cheapest; report the assignment, its total cost, the number of assignments evaluated and the message count, counting one status report from each agent and one instruction back to it.

```
(torch_env) D:\IMA lab>python lab6/lab6.py  
ALGORITHM 1 - CENTRALIZED ALLOCATION
```

Cost Matrix

	T	1T	2T	3T	4
A1		2	7	10	9
A2		1	4	7	6
A3		5	4	3	2
A4		10	5	2	5

Assignment:

A1 -> T1

A2 -> T2

A3 -> T4

A4 -> T3

Total Cost: 10

Assignments Evaluated: 24

Messages: 8

Algorithm 2 — Decentralized allocation by announced intentions

1. In each round, every agent that is still unassigned selects the cheapest task among those still free, breaking ties by task identifier, and broadcasts that intention to the other agents; count one message per recipient.
2. Resolve the round by giving each contested task to the agent that announced the lowest cost for it, with ties broken by agent identifier; mark those agents and tasks as committed and repeat with the agents that lost, until no agent or no task remains. Report the assignment, its total cost, the messages exchanged, the number of rounds and the bids announced in each round.

```

=====
ALGORITHM 2 - DECENTRALIZED ALLOCATION

Cost Matrix
      T   1T   2T   3T   4
A1      2   7  10   9
A2      1   4   7   6
A3      5   4   3   2
A4     10   5   2   5

Bids in each round:

Round 1:
A1 bids for T1 (cost = 2)
A2 bids for T1 (cost = 1)
A3 bids for T4 (cost = 2)
A4 bids for T3 (cost = 2)

Round 2:
A1 bids for T2 (cost = 7)

Final Assignment:
A1 -> T2
A2 -> T1
A3 -> T4
A4 -> T3
Total Cost: 12
Messages: 12
Rounds: 2

```

Procedure and Result

Paste the snapshots of the steps followed and the output produced by the program below, including the cost matrix, the assignment and total cost returned by each mechanism, the bids announced in every round of the decentralized protocol, and the message counts for both.

Observations

Comment on the difference in total cost between the two mechanisms and on which agent pays for it, on the round in which the decentralized protocol committed to a choice that turned out to be expensive, on how the message count of each mechanism grows with the number of agents, and on what each mechanism does when one agent stops responding.

Exercises

1. Generate random instances with the number of agents and tasks varying from three to eight, averaging over several instances at each size, and report the optimal cost, the decentralized cost, the message counts of both mechanisms and the number of assignments the coordinator evaluates. State the size at which exhaustive centralized search stops being practical, and the percentage by which the decentralized allocation exceeds the optimum.

EXERCISE 1 - RANDOM INSTANCES

Size	Optimal	Decentralized	Central Msg	Decentralized Msg	Evaluated	Excess %
3	32.20	32.60	6.00	6.80	6	1.24%
4	34.00	38.80	8.00	14.00	24	14.12%
5	50.40	54.80	10.00	21.20	120	8.73%
6	49.40	53.80	12.00	35.60	720	8.91%
7	64.20	68.20	14.00	50.40	5040	6.23%
8	63.40	67.40	16.00	70.80	40320	6.31%

2. Remove one agent before allocation begins and re-run both mechanisms; report the new assignment, the cost, and which task is left unserved. Then state what each mechanism does if the coordinator itself fails rather than an agent, and describe the change you would make to the centralized scheme to survive that failure.

EXERCISE 2 - REMOVE ONE AGENT

After removing A4:

Cost Matrix

	T	1T	2T	3T	4
A1		2	7	10	9
A2		1	4	7	6
A3		5	4	3	2

Centralized Allocation:

A1 -> T1

A2 -> T2

A3 -> T4

Total Cost: 8

Unserved Task: T3

Decentralized Allocation:

A1 -> T2

A2 -> T1

A3 -> T4

Total Cost: 10

Unserved Task: T3